

# Context

**A hands-on course in building agents by building their harness**

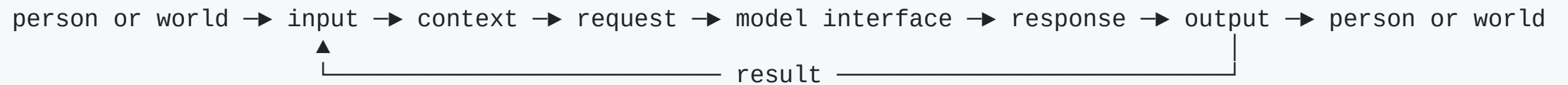
Lesson 4

# Every call starts from nothing

The model knows nothing about this moment except what's in the request:

- where it's running, or what day it is
- who it is
- what it did a minute ago
- what you told it last week

# Context decides what the request holds



- **Assembles:** chooses what goes in the request, and how it's presented
- **Fits:** when it's bigger than the context window, something has to give

# Almost everything is a context component

- **Instructions** and **self-knowledge**: who the model is, how it works
- **Memory**: working, episodic, semantic, procedural
- **Retrieval**: search a store, put what you found in the request
- **Compaction**: when working memory won't fit, summarize some of it

You need the ones your agent needs, built in whatever way fits.

## quark.py: Lesson 3, plus a context section

- 239 lines: 91 of code, and a system prompt of 148
- `messages` is renamed `working_memory` ; `append` becomes `add()`
- The call is wrapped in `try` , to catch "prompt is too long"
- The new section, context, has eight components

# Four memories: the harness records, the model distills

| Memory     | Who writes it                                             | Where                                            | How the model sees it                         |
|------------|-----------------------------------------------------------|--------------------------------------------------|-----------------------------------------------|
| Working    | the harness, <code>add()</code>                           | <code>working_memory</code>                      | every call, as the messages                   |
| Episodic   | the harness, <code>add()</code>                           | <code>.quark/episodes/&lt;start&gt;.jsonl</code> | by searching, when the past matters           |
| Semantic   | the model, with bash                                      | <code>.quark/memory/memory.md</code>             | by reading, when facts matter                 |
| Procedural | the model, with bash;<br>indexed by <code>skills()</code> | <code>.quark/skills/&lt;name&gt;.md</code>       | the index every call; a skill when it applies |

# The harness writes every message twice

```
EPISODE = f".quark/episodes/{datetime.datetime.now():%Y-%m-%dT%H-%M-%S}.jsonl"

def remember(message):
    os.makedirs(os.path.dirname(EPISODE), exist_ok=True)
    with open(EPISODE, "a") as f: f.write(json.dumps(message, ...) + "\n")

def add(working_memory, message):
    working_memory.append(message); remember(message)
```

(abridged: comments dropped, one long argument cut to ...)

# The system prompt is rebuilt every call

```
# Self Model
# Memory
## Semantic memory – facts that last
## Procedural memory – how to do things
## Episodic memory – what happened
## Across memories
# World Model
# Other Selves Model
# Body Operations
# Mechanics
```

- Rebuilt each call: the directory, the date and the skill index stay current
- The date but not the time, so the cached prompt stays identical
- `mechanics()` adds quark's own file, with `system()` redacted

# Semantic memory is a recipe in the prompt, not code

```
Format (preserve exactly; one fact per line, never two joined with ";" or "and"):  
- <subject>: <fact, phrased with the words future-you will grep for>
```

```
Write (the quoted heredoc keeps the fact literal):  
cat >> .quark/memory/memory.md << 'EOF'  
- <subject>: <fact>  
EOF
```

- **Distill:** the general truth behind what happened, not a record of it
- No time on a fact: when it was learned is episodic

# Procedural memory: skills, indexed every call

```
def skills():  
    index = []  
    for path in sorted(glob.glob(".quark/skills/*.md")):  
        ...  
    return "\n".join(index) or "- (none yet)"
```

(abridged)

- One Markdown file per skill, with a `name` and `description` at the top
- **Generalize:** the method behind a task, with placeholders, named for the class of task
- The index goes in the prompt; a whole skill is read only when a task matches

# Recall goes from most distilled to most complete

- **The ladder:** semantic, then procedural, then episodic
- Stop as soon as you have what you need
- Search all three stores at once
- Follow a fact to its skill, and a skill to the sessions that used it
- Find where a fact came from in the episodes: the only store with time

# Compaction is lazy

```
while True:
    try:
        if drop:
            working_memory, drop = compact(working_memory, drop), 0
            output = call(show, max_tokens=16384, system=system(), ...).content
    except BadRequestError as e:
        if "prompt is too long" not in str(e): raise
        drop += 1
        continue
```

(abridged)

- The API says "prompt is too long": `drop` goes up by one
- `compact()` drops the oldest `drop` turns and summarizes the rest
- The summary says where the originals are: the episode file

# It knows what it is because it's in context

```
uv run lessons/04-context/quark.py "what are you, and how do you work?  
three sentences"
```

I'm quark, an AI agent that works through a single tool, bash. That tool lets me observe and act on the whole system from the command line. I run a loop of observe, think, act, and repeat, and my context window is my working memory. Three stores outside it persist across sessions: semantic memory (facts), procedural memory (skills), and episodic memory (a log of every session). I read from them and write to the first two. The harness writes the third automatically.

# Facts outlast the session

"remember that I prefer short answers" → it appends `- user: prefers short answers`

A new run, "what do you know about me?" (shortened):

```
# Quark Memory
- user: prefers short answers
...
Not much. Here is what I know about you:
- You prefer short answers. This is saved in my memory.
- You've asked me how I work, and you asked me to remember your preference.
...
I don't know your name, your role, or anything else.
```

## It writes a skill for the whole class of task

"I'll often ask you how many lines of Python are in a repo, not counting `.venv`. Do it for this one now."

It answers, then saves this to `.quark/skills/count-python-lines.md` (shortened):

```
---
name: count-python-lines
...
1. cd to the repo root (default: the working directory).
2. Run: find . -name '*.py' -not -path '*/.venv/*' -print0 | xargs -0 cat | wc -l
...
4. Report total lines and file count.
```

## The next session reads the skill first

"how many lines of Python are in this repo?" (shortened)

```
$ cat .quark/skills/count-python-lines.md 2>/dev/null
---
name: count-python-lines
...
3335
25
```

The repo has **3,335 lines of Python** across **25 .py files**. I left out anything under **.venv**.

# Only episodic memory knows what you asked

`"what have I asked you in earlier sessions?"` (shortened)

In earlier sessions you asked me:

1. `"what are you, and how do you work? three sentences"`
2. `"remember that I prefer short answers"`
3. `"what do you know about me?"`
- ...
5. `"how many lines of Python are in this repo?"`

# Context can be built many ways

- **Who decides what's remembered:** the model, or harness code
- **What it retrieves:** nothing, or documents, code, past messages
- **When it fits:** after the API refuses, or before, by counting tokens
- **How it fits:** summarize old turns, drop them, or cut long tool results
- **How it's laid out:** one prompt, or instructions loaded when needed

Memory layers like [Mem0](#) are this primitive alone.

## context.py makes the other choices

- `fit()` counts tokens before every call and summarizes past `LIMIT`
- `trim()` keeps the start and end of a long tool result
- One shared episode log, `.quark/episodes.jsonl`; skills listed by first line

With `LIMIT` set to 560 tokens, after three summaries (shortened):

```
> [working memory over 560 tokens: summarized 3 messages]
Your favorite color is green. You told me that earlier in this session.
```

# The rule: assemble what the request holds, and fit it

- Instructions, self-knowledge, memory, retrieval and compaction all do that
- Who writes each memory is a choice
- The harness writes what happened; the model writes what it distills

## What context never does

- Get the request to the model and the response back: **model interface**
- Decide when to call, and whether a result goes back around: **control flow**
- Gather what goes in: **input**
- Handle the response: **output**

Context only decides what the request holds and how it fits.

# You've built a harness. Now take one apart.

|                    |                                                                 |
|--------------------|-----------------------------------------------------------------|
| control flow       | what kind of loop? who decides when to stop?                    |
| └─ input           | where do inputs come from: people, the world, both?             |
| └─ context         | what does the request hold? which memories? how does it fit?    |
| └─ model interface | where does the request go, and how does the response come back? |
| └─ output          | where do outputs go? which tools, and how are they run?         |

- Start with [nanoagent](#), or pick a big one
- Ask what each piece does until it fits one of the five

# **Next: Sandboxing**

Lesson 5